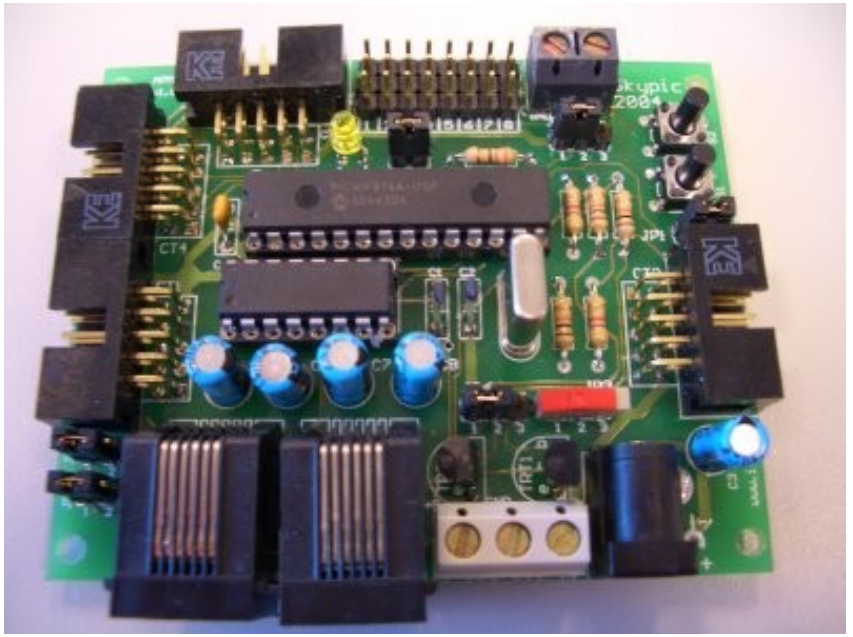
PIC 16F87X



Juan González

Escuela Politécnica Superior
Universidad Autónoma de Madrid

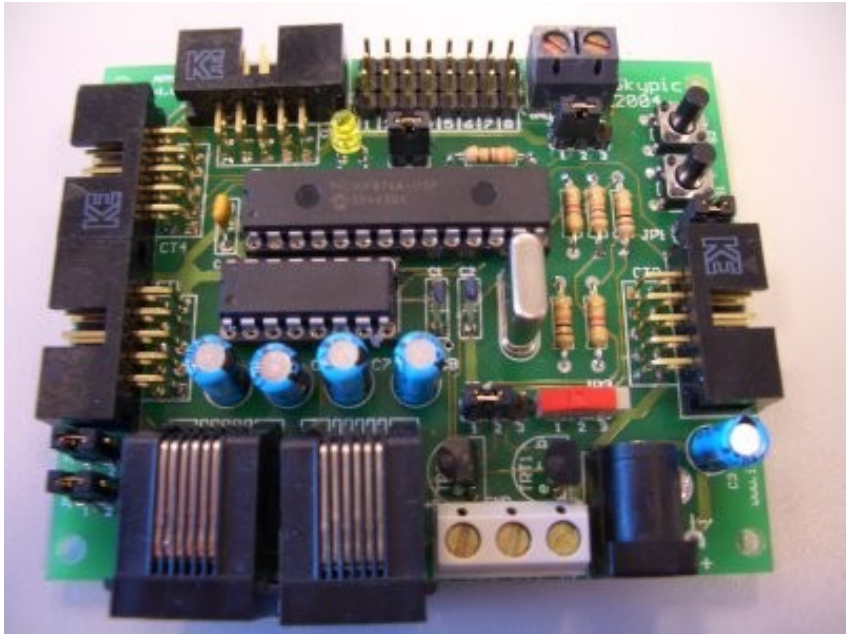
Andrés Prieto-Moreno

Flir Networked Systems

Ricardo Gómez

Flir Networked Systems

PIC 16F87X

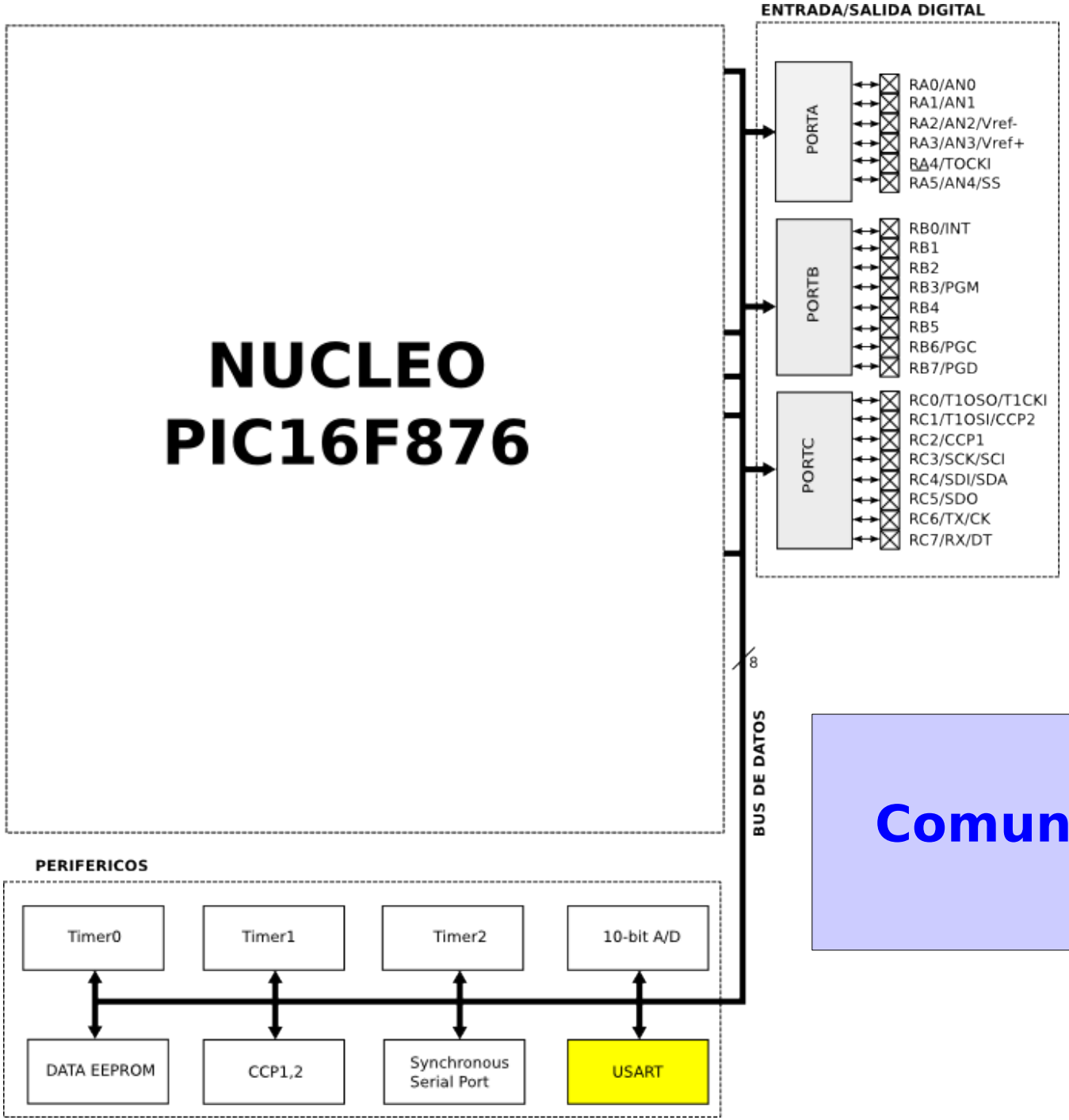


MÓDULO 3:

Puerto serie (SCI)

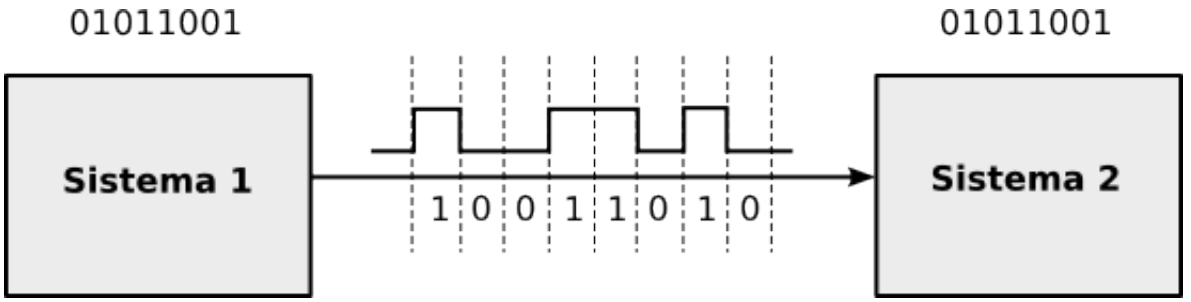
Introducción al lenguaje C (II)

Programación de periféricos



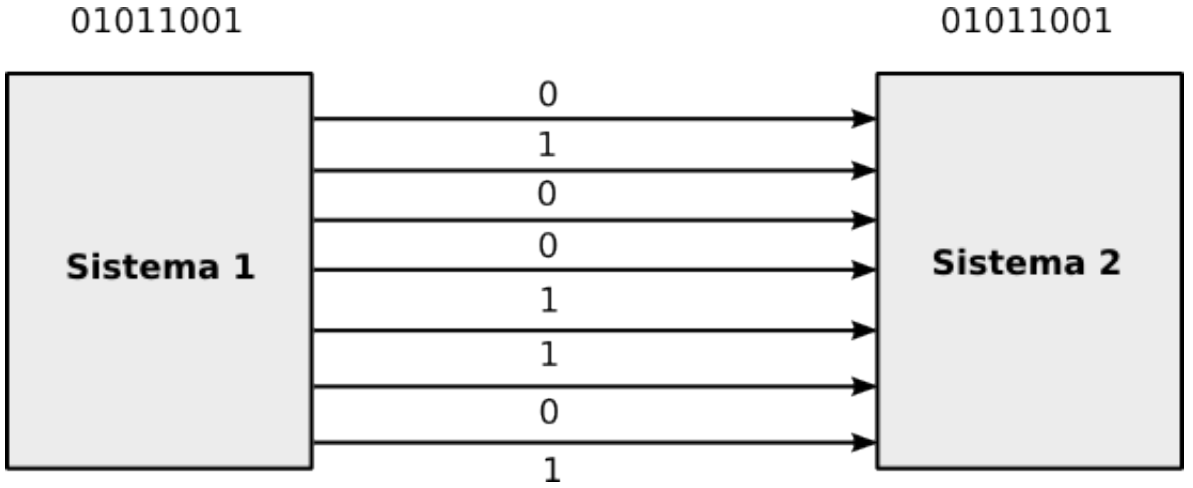
Comunicaciones serie

Comunicación serie VS paralela



SERIE

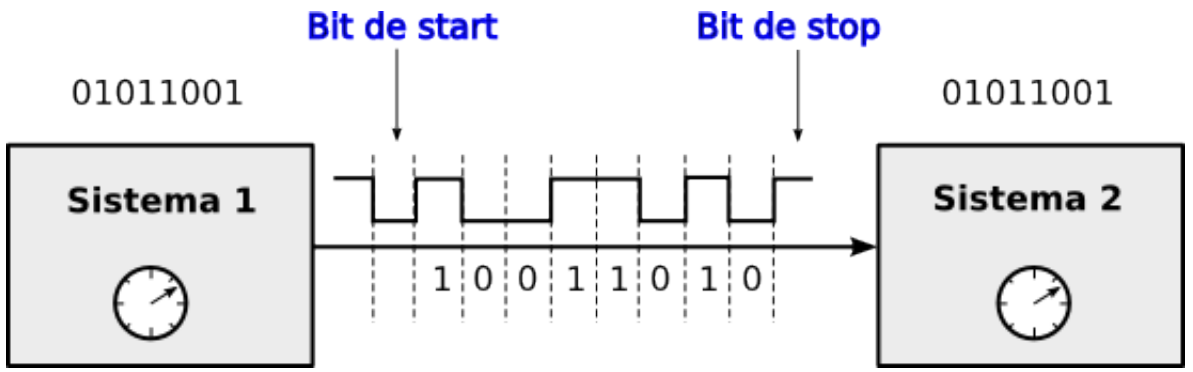
- Los bits se transmiten en serie, uno detrás de otro
- Sólo se necesita 1 hilo
- Más lento



PARALELO

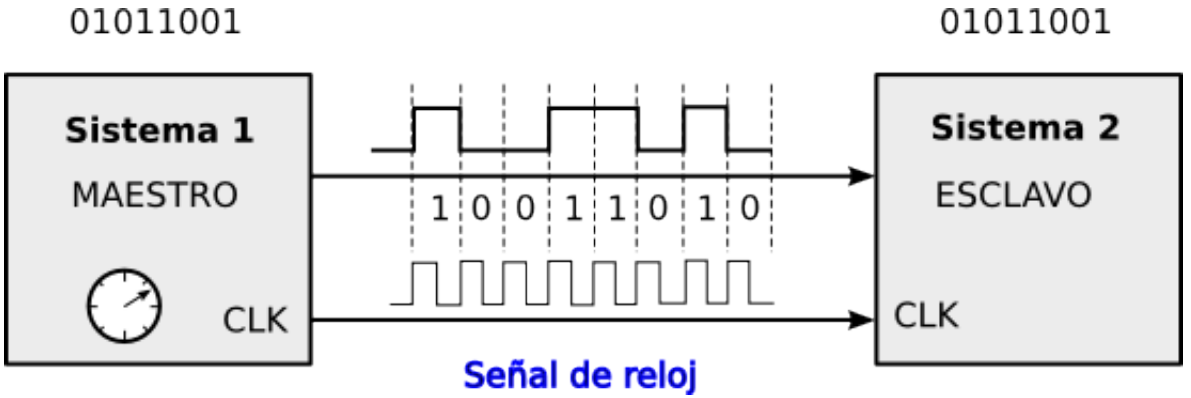
- Todos los bits se transmiten a la vez
- Necesarios 8 hilos
- Más rápido

Comunicación serie síncrona VS asíncrona



ASÍNCRONA

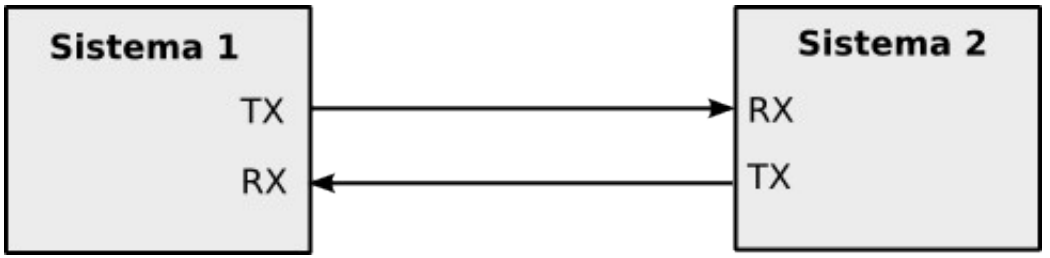
- Cada sistema tiene su propio reloj local.
- Sincronización mediante bits de start y Stop
- Sólo 1 hilo



SÍNCRONA

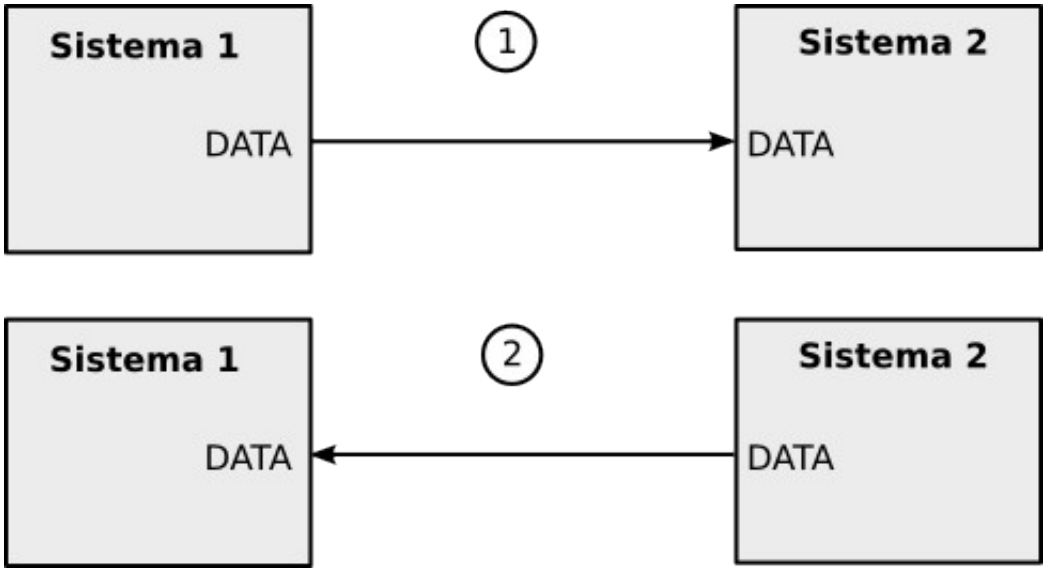
- Una señal de Reloj común
- El MAESTRO genera el reloj
- Dos hilos
- Velocidades mayores

Full-duplex VS Half-duplex



FULL-DUPLEX

- Comunicación bidireccional simultánea
- Dos canales de datos (TX, RX)
- Dos hilos



HALF-DUPLEX

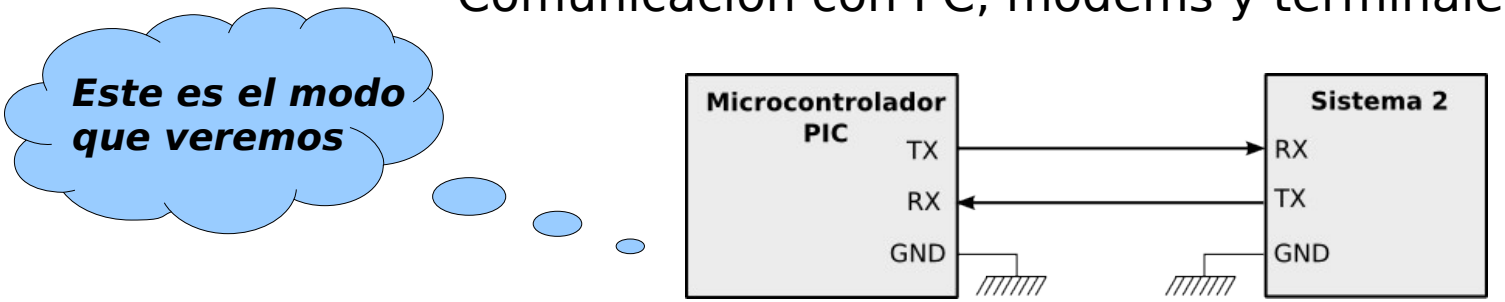
- Comunicación bidireccional multiplexada en el tiempo
- Un único canal (DATA)
- Primero en un sentido, luego en el otro (Protocolo)
- Un hilo

La USART del PIC

- **USART** = **U**niversal **S**ynchronous **A**synchronous **R**eceiver **T**ransmitter
- Otro nombre equivalente: **SCI**. **S**erial **C**ommunication **I**nterface

- **Modos de funcionamiento:**
  - Comunicación **serie Asíncrona** full-duplex

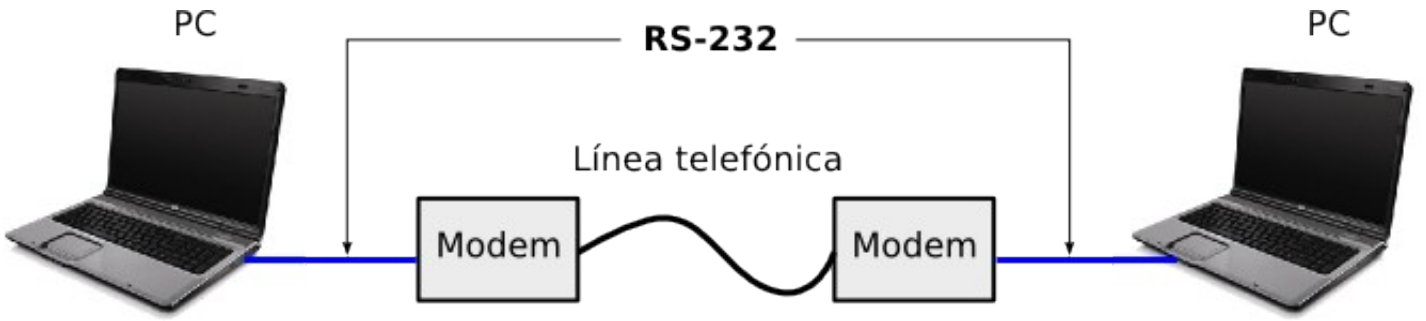
Comunicación con PC, modems y terminales



  - Comunicación **serie Síncrona** half-duplex (Maestro)
  - Comunicación **serie Síncrona** half-duplex (Esclavo)

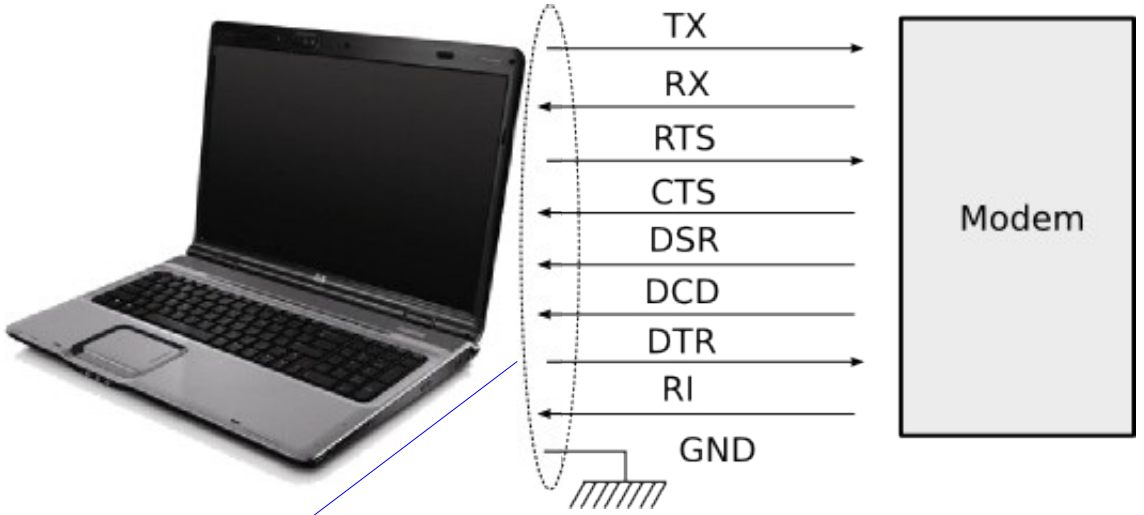
Memorias eeprom serie, conversores A/D, etc.

Norma RS-232 (I)



- Normaliza la comunicación entre el PC y el Modem
- Establece el protocolo, señales de control, tensiones, conectores, etc.
- La comunicación PC-modem es **serie asíncrona full-duplex**
- Esta comunicación se denomina vulgarmente “puerto serie”

Norma RS-232 (II)



DB9 macho

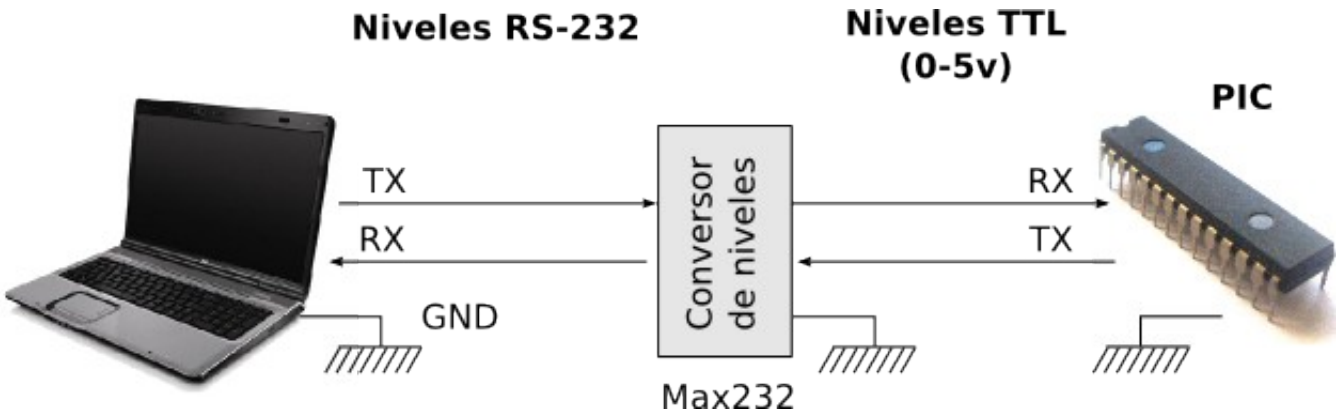


USB-serie



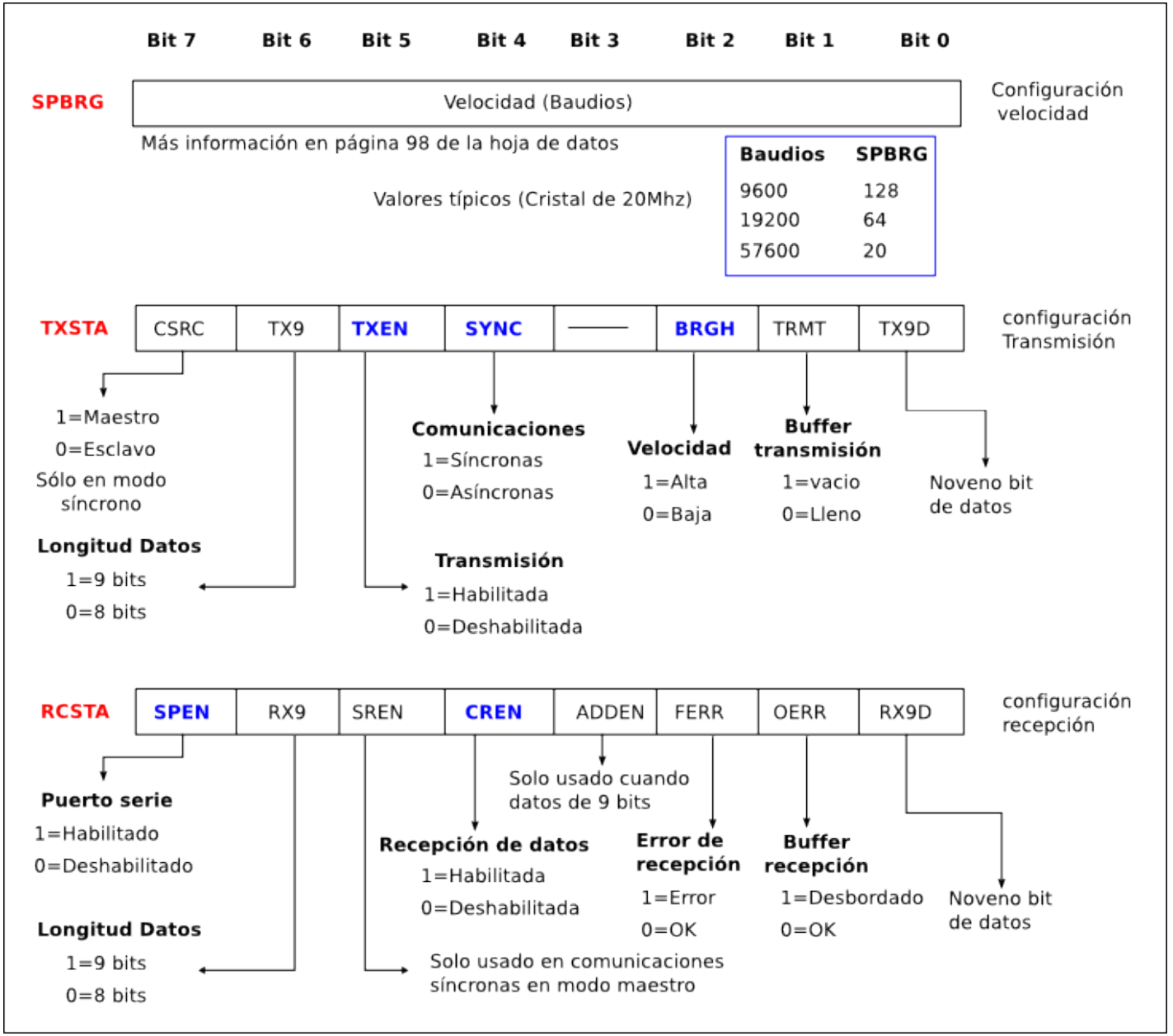
- Comunicación mediante **9 hilos**:
  - 2 de datos (TX, RX)
  - 8 de control
  - GND
- Niveles de tensión **NO TTL**: +- 15/12

Comunicación entre el PIC y el PC

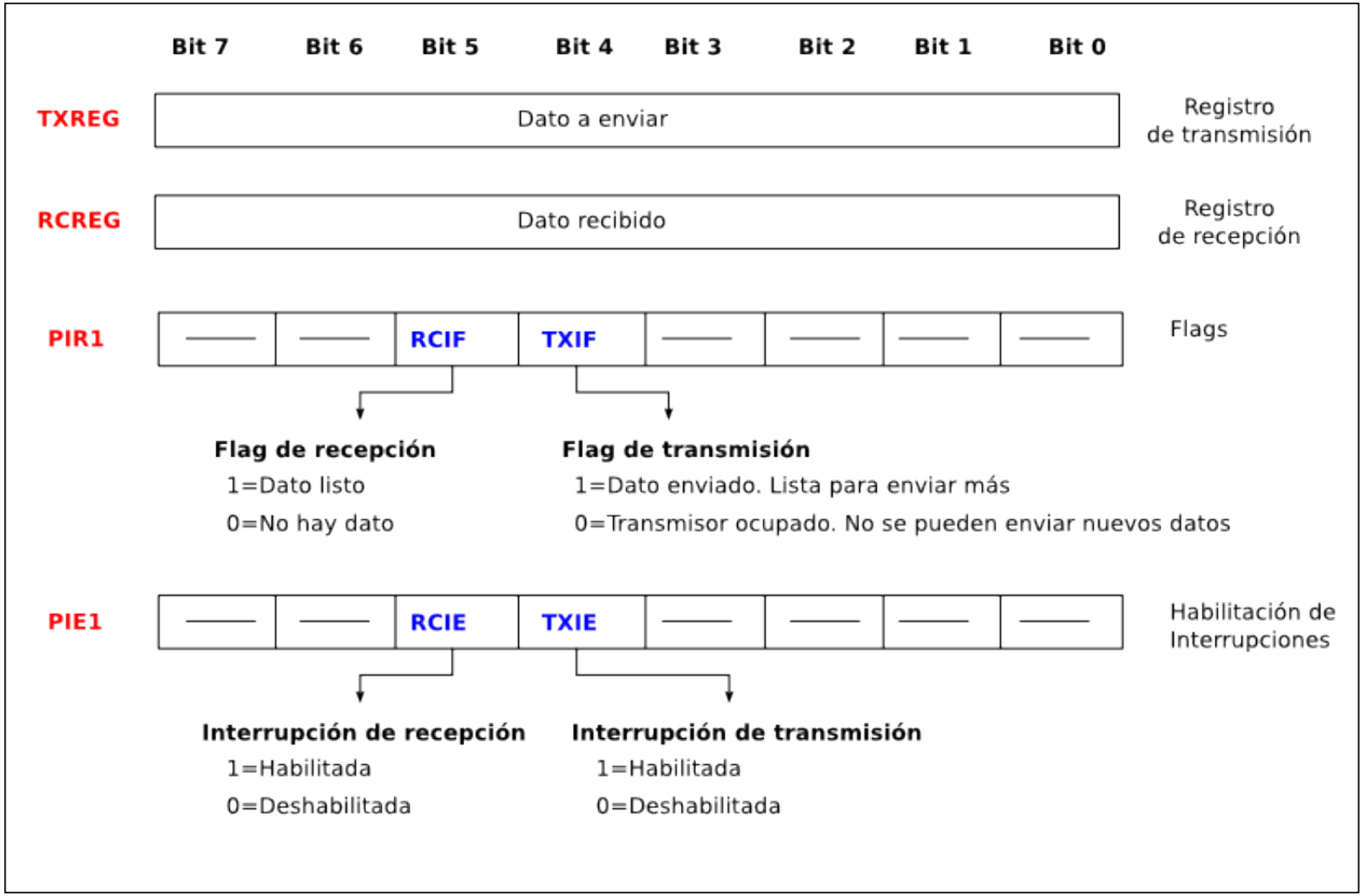


- Comunicación **serie asíncrona full-duplex**
- 2 hilos para datos: TX y RX
- Necesario **adaptar los niveles** de tensión. Ej. Chip max232
- Formato típico de transmisión: **8N1**:
  - 8 bits de datos
  - 1 bit de Stop / 1 bit de Start
  - Sin paridad
- Velocidades estandarizadas (en baudios):
  - 1200, 2400, **9600**, 19200, 38400, 57600, 115200

Registros del SCI en el PIC (I)



Registros del SCI en el PIC (II)



Ejemplo: Configuración del SCI

sci-conf.c

```
#include <pic16f876a.h>
#define LED RB1

void main(void)
{
  volatile unsigned char c;

  TRISB1=0;
  BRGH=1; SPBRG=129;
  SYNC=0;
  SPEN=1;
  CREN=1;
  TXEN=1;

  while(1) {
    while (RCIF==0);
    c=RCREG;
    LED^=1;
  }
}
```

Configuración a 9600 baudios, 8N1

Cada vez que se recibe un carácter se cambia el estado del led

Configurar LED

Configurar a 9600 baudios

Comunicaciones asíncronas

Activar pines del puerto serie

Habilitar el recepctor

Habilitar el transmisor

Esperar a que se reciba un byte

Leer byte recibido para que el flag RCIF se ponga a cero (volatile!)

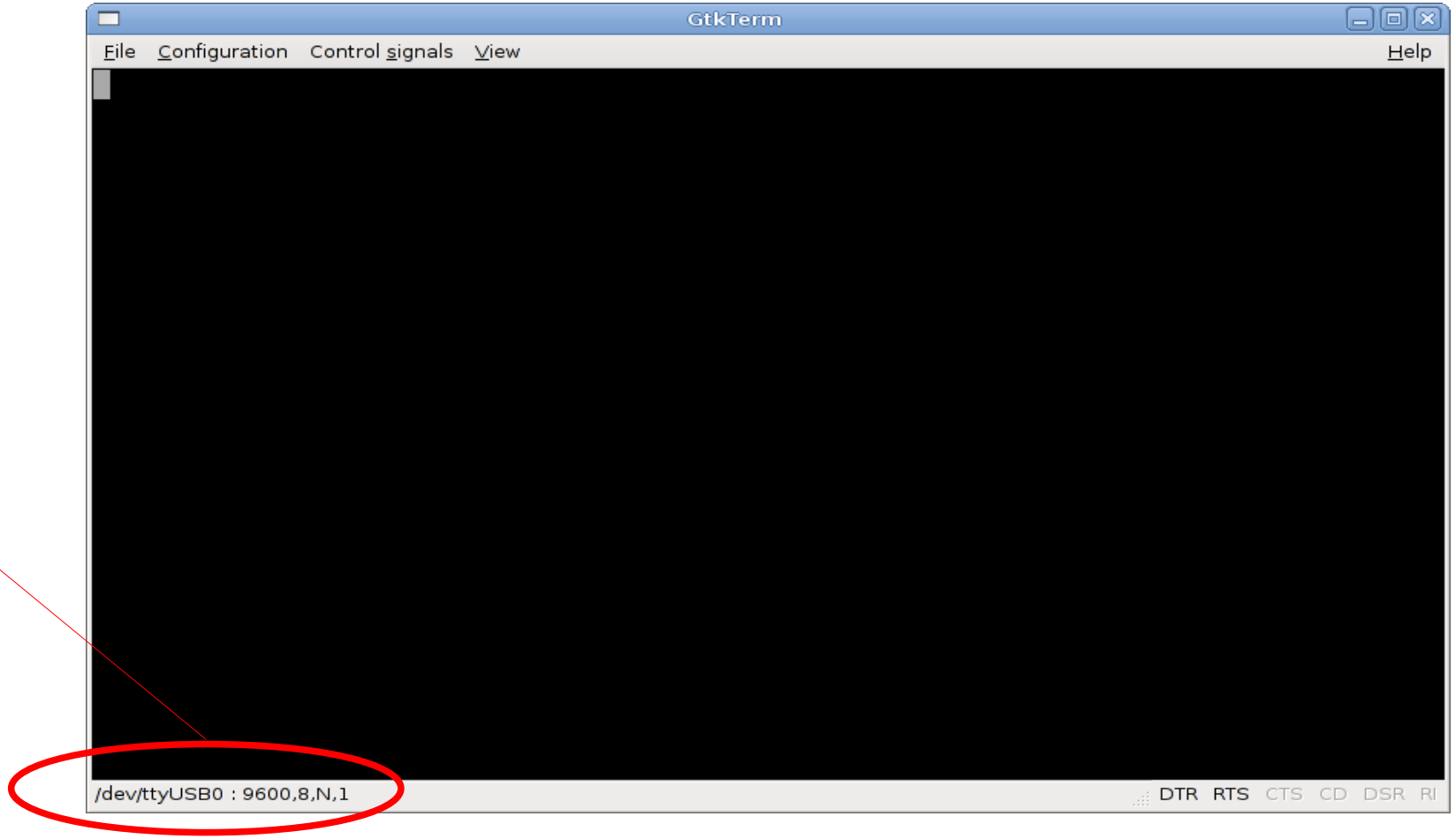
Cambiar el led de estado

Ejemplo: Configuración del SCI (II)

Para **probar** los ejemplos del puerto serie, necesitamos ejectuar en el PC un terminal de comunicaciones que funcione a 9600 baudios, y configurado a 8N1

- **Windows**: Hyperterminal
- **Linux**: Minicom, GTKTerm:

Configuración



Ejemplo: Configuración del SCI (III)

sci-conf2.c

```
#include <pic16f876a.h>
#define LED RB1

void sci_conf()
{
  BRGH=1;  SPBRG=129;
  SYNC=0;  SPEN=1;
  CREN=1;  TXEN=1;
}

void main(void)
{
  volatile unsigned char c;

  TRISB1=0;
  sci_conf();
  while(1) {
    while (RCIF==0);
    c=RCREG;
    LED^=1;
  }
}
```

Creamos una función para la configuración del sci. Esto nos permitirá reutilizar código y estructurar mejor los programas

Configurar puerto serie

Ejemplo: Envío-recepción de bytes

sci-eco.c

```
#include <pic16f876a.h>
#define LED RB1

void sci_conf() {...}

void main(void)
{
  unsigned char c;
  TRISB=0;
  sci_conf();

  while(1) {
    while (RCIF==0);
    c=RCREG;
    PORTB=c;
    while (TXIF==0);
    TXREG=c;
  }
}
```

Programa que hace “eco” de todo lo recibido desde el PC. Además, el byte recibido se muestra por los leds

¡Ya no hace falta que sea volatile!

Configurar puerto serie

Esperar a que llegue un carácter

Leerlo

Sacarlo por los leds

Transmisión. Primero hay que esperar a que el transmisor esté disponible

Enviar el carácter recibido

Como la variable c ahora se está usando, el optimizador no la elimina. No hace falta que sea del tipo volatile.

Ejemplo: Envío-recepción de bytes (II)

```
#include <pic16f876a.h>

unsigned char sci_read()
{
  while (RCIF==0);
  return RCREG;
}
void sci_write(unsigned char dato)
{
  while (TXIF==0);
  TXREG=dato;
}

void main(void)
{
  unsigned char c;

  TRISB=0;
  sci_conf();
  while(1) {
    c=sci_read();
    PORTB=c;
    sci_write(c);
  }
}
```

sci-eco2.c

Función para leer del sci

Función para enviar datos por el sci

Leer carácter

Enviar carácter al PC

Es igual que el ejemplo anterior, pero el utilizar funciones lo hace más legible, además de estar mejor estructurado

Cadenas de caracteres en C

• Los caracteres en C son del tipo unsigned char. Para asignar caracteres **ASCII** se usan las comillas simples ' '

```
unsigned char car='a';
```

• Algunos caracteres especiales:

'\n' : Nueva linea

'\r' : Retorno de carro

'\b': Back

• Las cadenas de caracteres en C son **arrays** del tipo unsigned char

• Por definición las cadenas en C **SIEMPRE DEBEN TERMINAR** con el valor 0

```
unsigned char cad[]={'h','o','l','a',0};
```

• Las cadenas en C se escriben entre **comillas dobles** “ “. El compilador añade automáticamente el 0 final:

```
unsigned char cad[]=”hola”;
```

¡¡Tiene un tamaño de 5 bytes!!

Ejemplo: Envío de cadenas

```
#include <pic16f876a.h>

void sci_conf() {...}
unsigned char sci_read() {...}
void sci_write(unsigned char dato) {...}

void sci_cad(unsigned char *cad)
{
  unsigned char i=0;

  while (cad[i]!=0) {
    sci_write(cad[i]);
    i++;
  }
}

void main()
{
  sci_conf();
  while(1) {
    sci_cad("Podemos!!!");
    sci_read();
  }
}
```

sci-cad.c

Funciones de los ejemplos anteriores

Nueva función para transmitir una cadena

Indice para recorrer la cadena

Recorrer la cadena. El último elementos es un 0

Transmitir el carácter i

Apuntar al siguiente carácter

Enviar la cadena al PC. Se usan comillas.

Esperar a que reciba un carácter

Programa que envía una cadena cada vez que se recibe un carácter por el puerto serie

Ejemplo: Envío de cadenas (II)

sci-cad2.c

```
#include <pic16f876a.h>

void sci_conf() {...}
unsigned char sci_read() {...}
void sci_write() {...}
void sci_cad() {...}

void main(void)
{
  unsigned char cad[]="hola\b\b\b\b";

  sci_conf();
  TRISB=0;

  while(1) {
    sci_cad(cad);
    PORTB=sci_read();
  }
}
```

Funciones de los ejemplos anteriores

Cadena que contiene caracteres especiales

Enviar la cadena

Esperar a que llegue un carácter y sacarlo por los leds, para mostrar que hay actividad

La instrucción switch

- **Sintáxis**:

```
switch(variable) {
   case valor1:
      Instrucciones
      break;
   case otro_valor:
      Instrucciones
      break;
   [...]
   default:
      Instrucciones
}
```

- **Se utiliza para realizar diferentes acciones en función de la variable pasada como argumento**
- **Muy útil para hacer “menus”**

Se ejecutan esas instrucciones cuando se cumple que variable=valor1

Se ejecutan esas instrucciones cuando se cumple que variable=otro_valor

Se ejecutan esas instrucciones si no se ha cumplido ninguno de los casos anteriores (Esta parte es opcional)

La instrucción switch: Ejemplo (I)

sci-menu.c (parte 1)

```
#include <pic16f876a.h>
#define LED RB1
void sci_conf() {...}
unsigned char sci_read() {...}
void sci_write() {...}
void sci_cad() {...}

void menu(void)
{
  sci_cad("Menu\n");
  sci_cad("----\n");
  sci_cad("1.- Cambiar el estado del led\n");
  sci_cad("2.- Poner todo el puerto B a 1\n");
  sci_cad("3.- Reset del puerto B\n");
  sci_cad("4.- Sacar este menu otra vez\n");
  sci_cad("Opcion? ");
}
```

Funciones de los ejemplos anteriores

Función para sacar el menú

Carácter \n al final de cada cadena: Salto de línea

Si se prueba desde el hyperterminal de windows posiblemente haya que añadir \r al final, para que se vea correctamente

La instrucción switch: Ejemplo (II)

sci-menu.c (parte 2)

```
void main(void)
{
  unsigned char c;
  TRISB=0;
  sci_conf();
  menu();
  while(1) {
    c=sci_read();
    switch(c) {
      case '1':
        LED^=1;
        break;
      case '2':
        PORTB=0xFF;
        break;
      case '3':
        PORTB=0x00;
        break;
      case '4':
        menu();
        break;
    }
  }
}
```

Sacar el menú

Esperar la opción del usuario

Según el carácter recibido....

Si es la opción 1...

Cambiar el led de estado

Si es la opción 2...

Todos los pines del puerto B a '1'

Si es la opción 3...

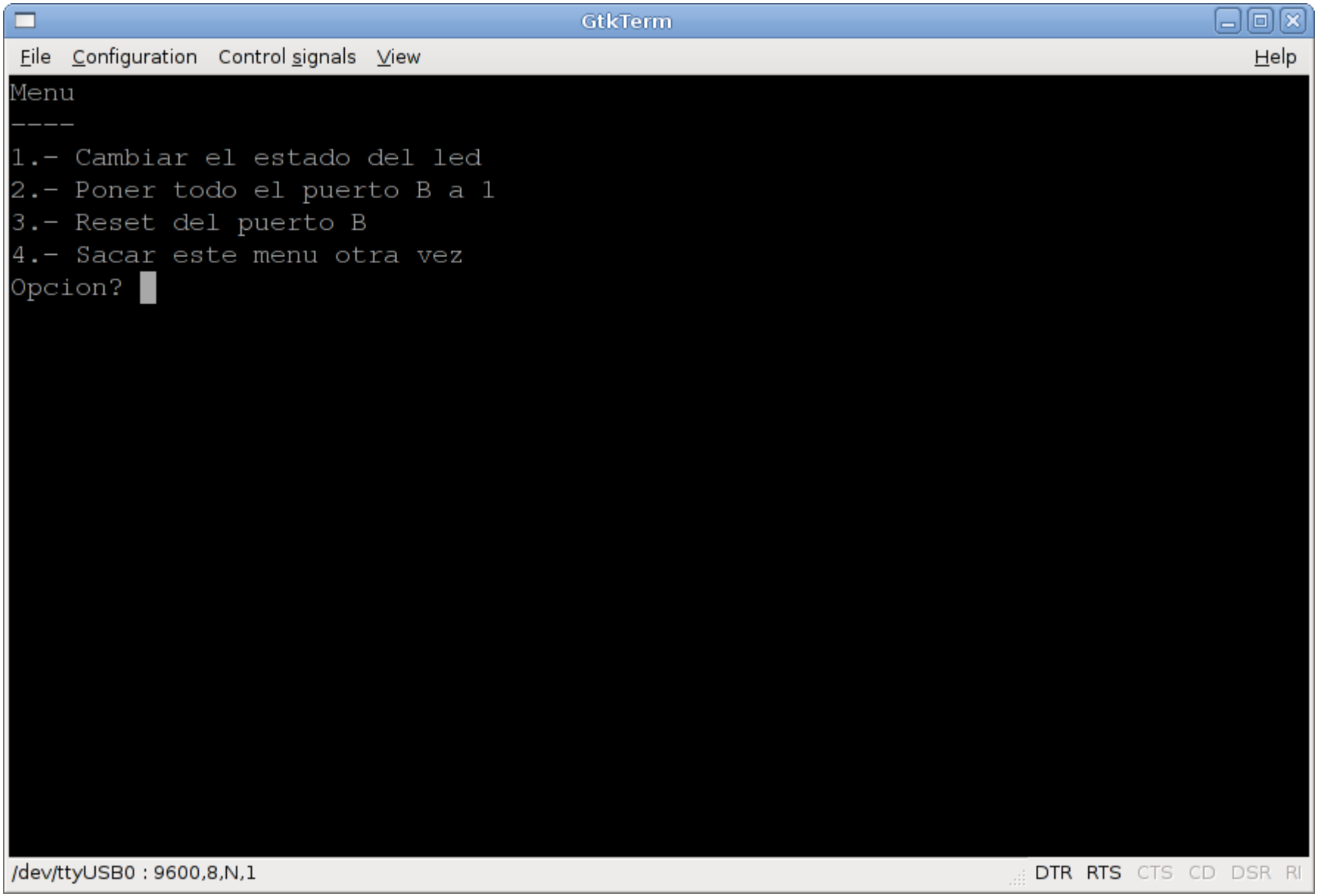
Todos los pines del puerto B a '0'

Si es la opción 4...

Sacar el menú

Probando el programa sci-menu.c

- Arrancar el terminal. Pulsar reset. Aparecerá el menú:



Ejercicio:

• Hacer un programa que **envíe una cadena** (por ejemplo “Hola como estas...”) por el puerto serie, pero **carácter a carácter**.

• Al arrancar se envía el primer carácter. Cuando **el usuario pulsa una tecla**, se envía el siguiente. Al volver a pulsar otra tecla, se envía el siguiente. Y así sucesivamente.

• Cuando se haya enviado la cadena completa se comenzará otra vez desde el principio

Errores en la recepción

- Error en dato recibido
  - Se detecta porque el **bit de STOP recibido es incorrecto**. Siempre tiene que ser '1'. Si se recibe un '0' es debido a un error
  - Se puede producir por **ruido en la línea** o por des-sincronización (por ejemplo porque las velocidades entre el PC y el PIC sean diferentes)
  - Se activa el flag FERR. El flag se pone a 0 si se lee RCREG y llega un dato nuevo.
- Acciones a tomar:
  - El software de bajo nivel no puede hacer nada
  - Hay que notificarlo al software de nivel superior para:
    - Que se descarte el dato
    - Que se solicite retransmisión (PROTOCOLO)
  - Este error se puede ignorar si se están empleando tramas que llevan incorporado un CRC para la detección de errores.

Errores en la recepción (II)

- Error de overflow
  - Se produce cuando el buffer de recepción (2 bytes) se llena y el software del PIC no ha realizado lecturas.
  - Esto ocurre si el software no lee los datos recibidos con la suficiente frecuencia
  - El efecto es que **se pierden datos**.
  - Se activa el flag OERR.



La recepción serie se DESACTIVA. Los nuevos datos recibidos se descartan hasta que el software ponga el flag OERR a 0



- Acciones a tomar:
  - Para desactivar OERR, hay que poner CREN=0 y luego CREN=1. Con esto se activa el recptor y se pueden seguir recibiendo datos
  - Igual que en el caso anterior, se puede notificar al nivel superior para solicitar retransmisión de datos (PROTOCOLO).
  - Se puede ignorar si se están empleando tramas que llevan incorporado un CRC para la detección de errores, o bien se están usando **timeouts**.

Ejemplo de overflow

sci-error.c

```
#include <pic16f876a.h>
#define LED RB1
void sci_conf() {...}
unsigned char sci_read() {...}
void sci_write() {...}
void sci_cad() {...}
void pausa() {...}

void main(void)
{
  TRISB=0;
  sci_conf();
  while(1) {
    sci_cad("Hola... ");
    pausa(4);
    sci_read();
    LED=OERR;
  }
}
```

Funciones de los ejemplos anteriores

Enviar cadena al PC

Pausa. Simulamos que el PIC está haciendo operaciones

Leer un dato por el puerto serie

Mostrar el flag OERR por el LED. Si se enciende, es que ha ocurrido un overflow

Si se envían caracteres desde el terminal a baja frecuencia, no hay overflow... pero en cuanto tocamos las teclas más rápido... ¡¡Este programa tan simple deja de funcionar!!

¿Dónde se cuelga?

Ejemplo de overflow (II)

- Es el típico fallo **DIFÍCIL DE ENCONTRAR**
- Al producirse el overflow, el receptor se desactiva... y el flag RCIF se pone a 0.
- Como ya no llegan datos nuevos (receptor desactivado), RCIF NUNCA SE PONE A '1'.
- Por tanto, la instrucción `while (RCIF==0);` que se encuentra en la función sci_read() se convierte en UN BUCLE INFINITO!!!

Principio del while(): Los bucles del tipo while(flag==valor); son POTENCIALEMENTE PELIGROSOS. Si nuestro programa se cuelga, es lo primero que hay que mirar

Solución:

- Usar while comprobando al menos dos condiciones: el valor que queremos y la condición de error o un timeout, de manera que nunca pueda entrar en bucle infinito
- Usar interrupciones.

Solución para el bloqueo del programa sci-error.c

El problema se soluciona reescribiendo la función sci_read()

sci-read2.c

```
unsigned char sci_read()
{
  unsigned char car;

  while (RCIF==0 && OERR==0);
  car = RCREG;
  if (OERR==1) {
    CREN=0;
    CREN=1;
    //-- Accion a tomar
    ...
  }
  return car;
}
```

Nueva condición. Se espera hasta que llegue un dato o bien se produzca un error de overflow

Leer carácter recibido

Si ha ocurrido un error...

Poner el flag OERR a 0 reiniciando el transmisor

Realizar la acción correctora pertinente. Esto depende de la aplicación. En este ejemplo se ignora el error

El programa ahora no se cuelga

Operadores booleanos y condicionales

== Verdadero si se cumple la igualdad. Ej. RCIF==0

!= Verdadero si los valores son distintos. Ej. RCIF!=1

> Verdadero si el argumento izquierdo es mayor que el derecho

< Verdadero si el argumento izquierdo es menor que el derecho

>= Verdadero si el argumento izquierdo es mayor o igual que el derecho

<= Verdadero si el argumento izquierdo es menor o igual que el derecho

&& Operador booleano “Y” Ej. RCIF==0 && OERR==0

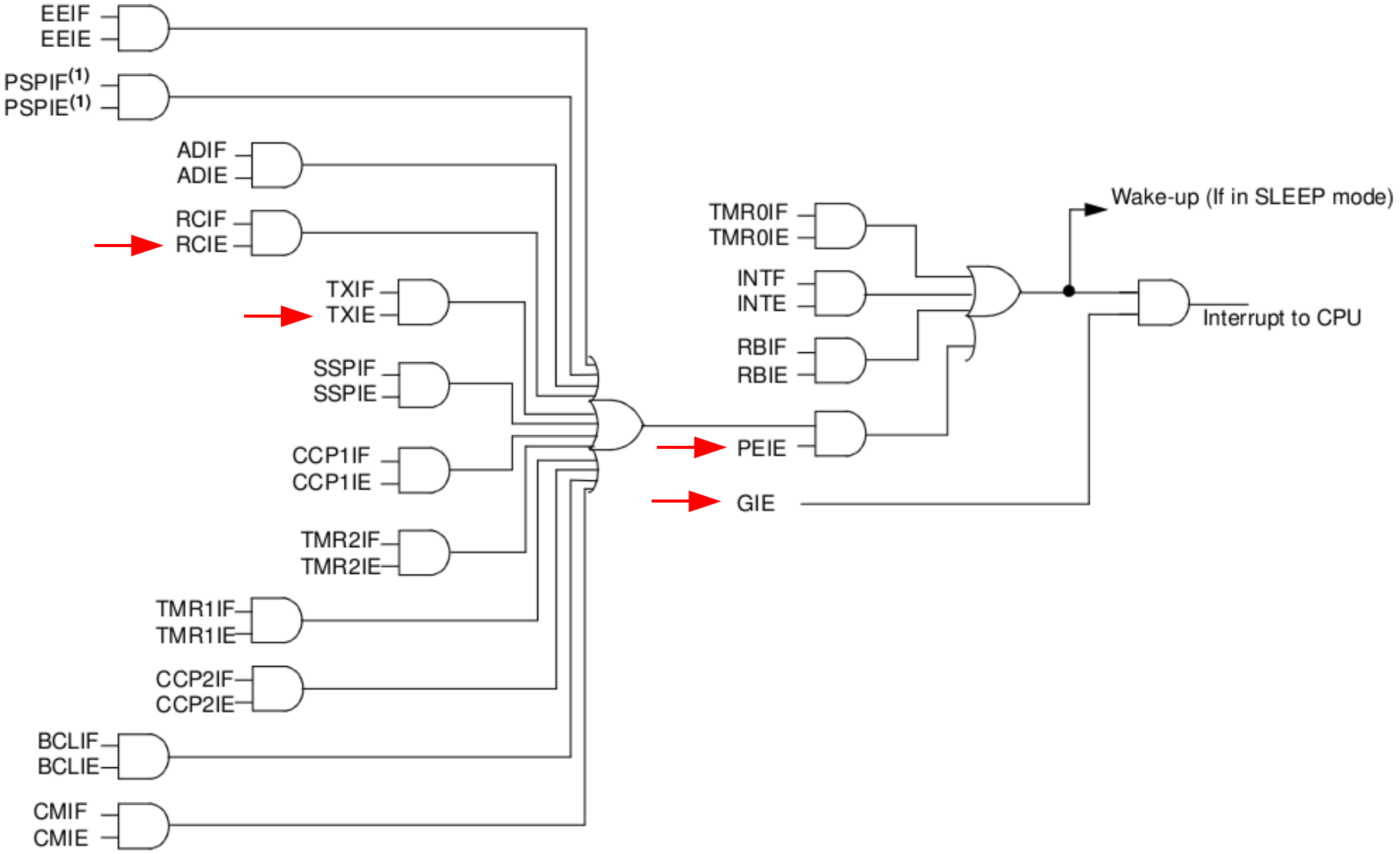
|| Operador booleano “O”

! Operador boleano NOT Ej. !(a==0 && b==1)

En C:

Verdadero significa que se devuelve un número positivo distinto de 0
Falso significa que se devuelve 0

Interrupciones



Note 1: PSP interrupt is implemented only on PIC16F874A/877A devices.

Interrupción de carácter recibido: Pruebas

sci-int-rx.c

```
#include <pic16f876a.h>
#define LED RB1

volatile unsigned char c;

void sci_conf(){..}

void isr() interrupt 0
{
  c=RCREG;
  LED^=1;
}

void main(void)
{
  TRISB=0;
  sci_conf();

  RCIE=1;
  PEIE=1;
  GIE=1;

  while(1);
}
```

Programa “hola mundo” para probar las interrupciones del puerto serie. Cada vez que se recibe un carácter se cambia de estado el led

Rutina de atención a la interrupción. Cada vez que se recibe un carácter se ejecuta

Borrar flag. Para ello hay que leer el dato recibido

Cambiar el led de estado

Configurar puerto serie

Activar interrupciones

El bucle principal no hace nada

Interrupción de carácter recibido

• Mediante el uso de interrupciones se pueden realizar tareas "simultaneamente"

• Vamos a hacer un programa que realice estas dos tareas sencillas e independientes:

  • Tarea 1: Un contador, que se visualiza en los leds del puerto B

  • Tarea 2: Hacer eco de todo lo recibido por el puerto serie

• La tarea 1 se realiza en el **bucle principal**

• Para la tarea 2 vamos a utilizar las interrupciones de dato recibido. Cada vez que se reciba un dato por el puerto serie, aparecerá la interrupción y nuestra **rutina de atención a la interrupción** lo leerá y lo reenviará

Interrupción de carácter recibido (II)

sci-contador-eco.c

```
#include <pic16f876a.h>

unsigned char c;

void sci_conf() {...}
void sci_write() {...}
void pausa() {...}

void isr() interrupt 0
{
  c=RCREG;
  sci_write(c);
}

void main(void)
{
  unsigned char cont=0;

  TRISB=0;
  sci_conf();
  RCIE=1;
  PEIE=1;
  GIE=1;
  while(1) {
    PORTB=cont;
    pausa(4);
    cont++;
  }
}
```

Tarea 2: Eco por el puerto serie

Leer carácter

Enviarlo al PC

Variable contador

Activar las interrupciones

Tarea 1: Contador que se muestra por los leds

Mostrar contador actual por los leds

Incrementar el contador

Interrupción de carácter transmitido: prueba

sci-int-tx.c

```
#include <pic16f876a.h>
unsigned char i=0;
unsigned char cad[]={'h','o','l','a',0};

void sci_conf() {...}

void isr() interrupt 0
{
  if (cad[i]!=0) {
    TXREG=cad[i];
    i++;
  }
}

void main(void)
{
  unsigned char cont=0;

  TRISB=0;
  sci_conf();
  TXIE=1;
  PEIE=1;
  GIE=1;
  while(1);
}
```

Programa “hola mundo” para probar las interrupciones del puerto serie. Se envia la cadena “hola” mediante interrupciones

Rutina de atención a la interrupción. Cada vez que el PIC está listo para enviar un carácter se ejecuta esta rutina

Activar interrupciones

El bucle principal no hace nada

Interrupción de carácter transmitido

- Otro ejemplo que realice dos tareas independientes
  - Tarea 1: Un contador, que se visualiza en los leds del puerto B
  - Tarea 2: Otro contador de 0 a 9 por el puerto serie. Se envian constantemente estos caracteres hacia el PC
- La tarea 1 se realiza en el **bucle principal**
- Para la tarea 2 utilizamos las interrupciones de listo para transmitir. Cada vez que se esté listo para enviar un dato, se ejecuta la **rutina de atención a la interrupción** que mandará el siguiente carácter.

Interrupción de carácter transmitido (II)

sci-contadores.c

```
#include <pic16f876a.h>
unsigned char i=0;

void sci_conf() {...}
void pausa() {...}

void isr() interrupt 0
{
  TXREG=i+'0';
  i=(i+1)%10;
}

void main(void)
{
  unsigned char cont=0;

  TRISB=0;
  sci_conf();
  TXIE=1;
  PEIE=1;
  GIE=1;
  while(1) {
    PORTB=cont;
    pausa(4);
    cont++;
  }
}
```

Tarea 2: Enviar los numeros del '0' al '9' constantemente

Enviar el siguiente dígito. La variable i contiene el numero en binario. Con la suma i + '0' se obtiene el carácter ASCII correspondiente a i. '0' es el ASCII del carácter '0'

Incrementar i. Con el operador % logramos que cuando la i llegue a 10 se le asigne el valor 0 para volver a comenzar la cuenta.

Activar las interrupciones

Tarea 1: Contador binario por los leds del puerto B

Mostrar contador actual por los leds

Incrementar contador

Reutilización de código

- Ya tenemos unas **funciones simples** para manejar el puerto serie
- Queremos poder **utilizarlas desde nuestros programas**, sin tener que estar haciendo “copy&paste”
- Queremos **independizarlas en su propio archivo** de forma que los cambios que hagamos se reflejen en todos los programas que las usen

Formas de hacerlo:

1) **Código en fichero .h**. Método “cutre” (pero efectivo).

2) **Crear un módulo**. El Código está en fichero .c. Método tradicional

3) **Creación de librería** (opcional). Agrupar varios módulos en una librería

Reutilización de código: Código en fichero .h (I)

• La manera **más sencilla** es poner todas las funciones a reutilizar en un fichero con extensión .h. Ejemplo: sci.h

• Al comienzo de nuestros programas usamos la instrucción:

```
#include "sci.h"
```

VENTAJAS:

• Método sencillo y rápido. Válido para hacer pruebas rápidas

• No hay que cambiar ningún parámetro de compilación

INCONVENIENTES:

• Método "cutre". Los fichero .h no se usan para incluir código, sino para almacenar macros, deficiones y los prototipos de las funciones.

• Cada vez que se compila nuestro programa, también se compilarán estas funciones, aunque no se hayan modificado. Si es un proyecto grande se pierde tiempo de compilación innecesariamente

Reutilización de código: Código en fichero .h (II)

sci.h

```
#include <pic16f876a.h>

void sci_conf()
{
  BRGH=1;  SPBRG=129;
  SYNC=0;  SPEN=1;
  CREN=1;  TXEN=1;
}

unsigned char sci_read()
{
  unsigned char car;

  while (RCIF==0 && OERR==0);
  car = RCREG;
  if (OERR==1) {
    CREN=0;
    CREN=1;
  }
  return car;
}

void sci_write(unsigned char dato)
{
  while (TXIF==0);
  TXREG=dato;
}
```

```
void sci_cad(unsigned char *cad)
{
  unsigned char i=0;

  while (cad[i]!=0) {
    sci_write(cad[i]);
    i++;
  }
}
```

- Todas las funciones del SCI las agrupamos en el fichero sci.h

Reutilización de código: Código en fichero .h (III)

sci-test.c

```
#include
<pic16f876a.h>
#include "sci.h"

void main(void)
{
  unsigned char c;

  TRISB=0;
  sci_conf();

  while(1) {
    c=sci_read();
    PORTB=c;
    sci_write(c);
  }
}
```

Incluir el archivo sci.h. El compilador lo incluye automáticamente y lo compila todo junto

Programa "clásico" del Eco, pero usando las funciones definidas en sci.h. El programa es ahora mucho más corto y fácil de entender.

Compilación:

sdcc -Wl-ainhx8m -mpic14 -p16f876a -o sci-test sci-test.c

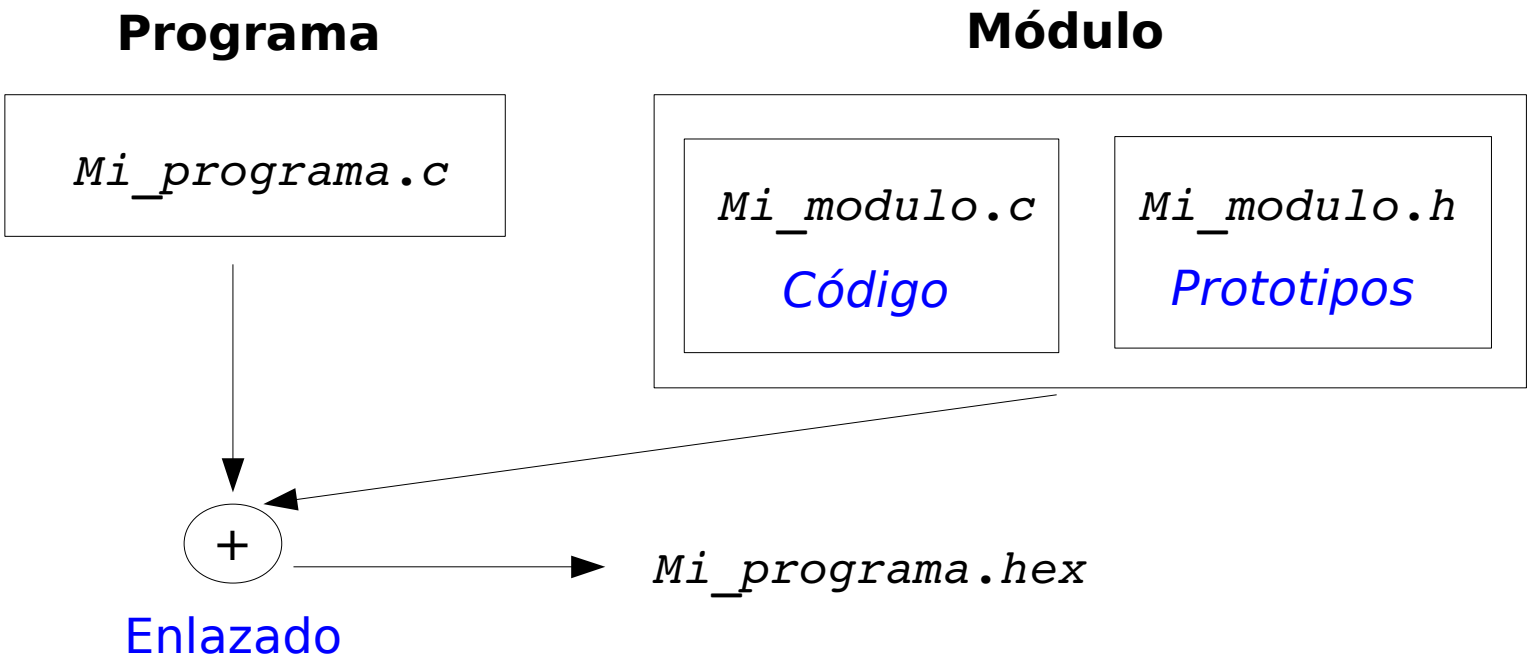
Exactamente igual que los ejemplos vistos hasta ahora

Reutilización de código: Módulos (I)

• La manera clásica de estructurar un programa en C es dividiendo el código en módulos.

• Cada módulo consta de un fichero .c con el código y un fichero .h con las definiciones de las funciones (prototipos) y las definiciones.

El programa que use el módulo creado **debe incluir el fichero de cabecera**: `#include "modulo.h"`



Módulo libsci para el manejo del SCI (I)

libsci.c

```
#include <pic16f876a.h>

void sci_conf()
{
  BRGH=1;  SPBRG=129;
  SYNC=0;  SPEN=1;
  CREN=1;  TXEN=1;
}

unsigned char sci_read()
{
  unsigned char car;

  while (RCIF==0 && OERR==0);
  car = RCREG;
  if (OERR==1) {
    CREN=0;
    CREN=1;
  }
  return car;
}

void sci_write(unsigned char dato)
{
  while (TXIF==0);
  TXREG=dato;
}
```

```
void sci_cad(unsigned char *cad)
{
  unsigned char i=0;

  while (cad[i]!=0) {
    sci_write(cad[i]);
    i++;
  }
}
```

libsci.h

```
void sci_conf();
unsigned char sci_read();
void sci_write(unsigned char dato);
void sci_cad(unsigned char *cad);
```

Prototipos

Módulo libsci para el manejo del SCI (II): Ejemplo

sci-test2.c

```
#include <pic16f876a.h>
#include "libsci.h"

void main(void)
{
  unsigned char c;
  TRISB=0;
  sci_conf();

  while(1) {
    c=sci_read();
    PORTB=c;
    sci_write(c);
  }
}
```

Programa “clásico” del Eco, pero usando el módulo libsci

Incluir el archivo libsci.h. El compilador puede saber si se están invocando con los parámetros correctos

Compilar el módulo libsci.c

Compilar el programa sci-test2.c

Enlazar los ficheros objeto y crear el ejecutable

Compilación:

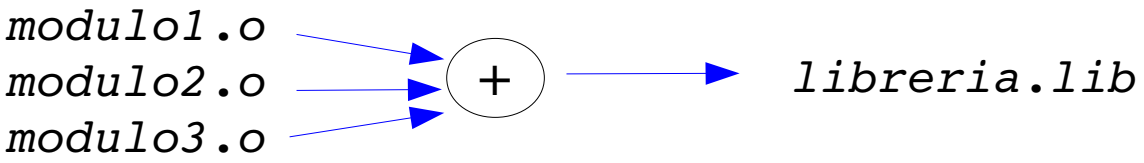
sdcc -mpic14 -p16f876a -c libsci.c

sdcc -mpic14 -p16f876a -c sci-test2.c

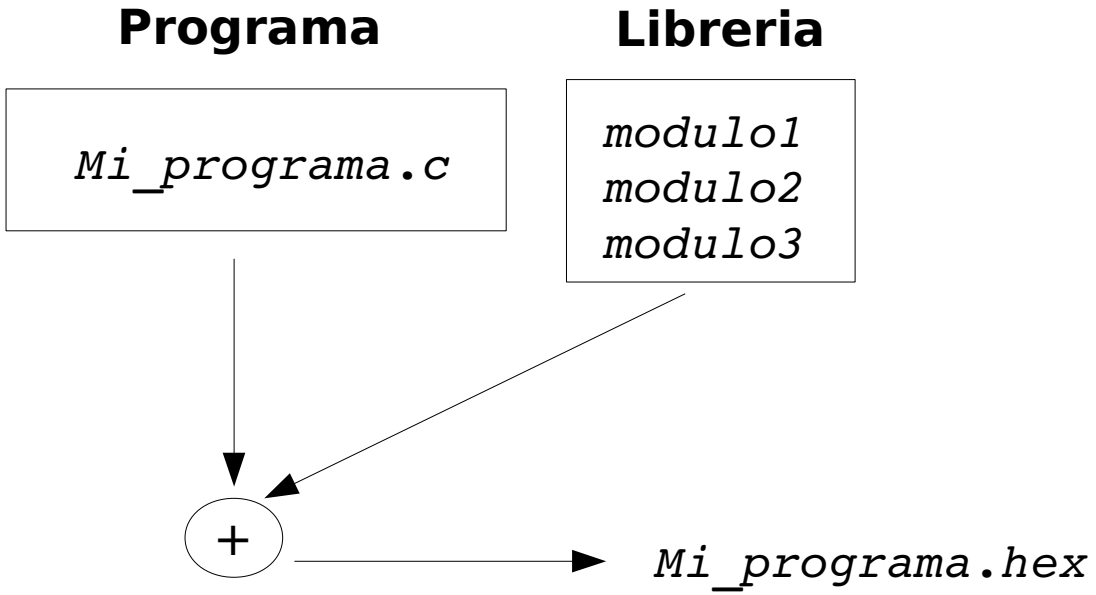
sdcc -Wl-ainhx8m -mpic14 -p16f876a -o sci-test2 libsci.o sci-test2.o

Reutilización de código: Librerías (I)

• Con las librerías agrupamos diferentes módulos en un único fichero .lib



• Los programas se enlazan directamente con la librería para crear los ejecutables:



Enlazado

Librerías (II). Ejemplo: libsci.lib

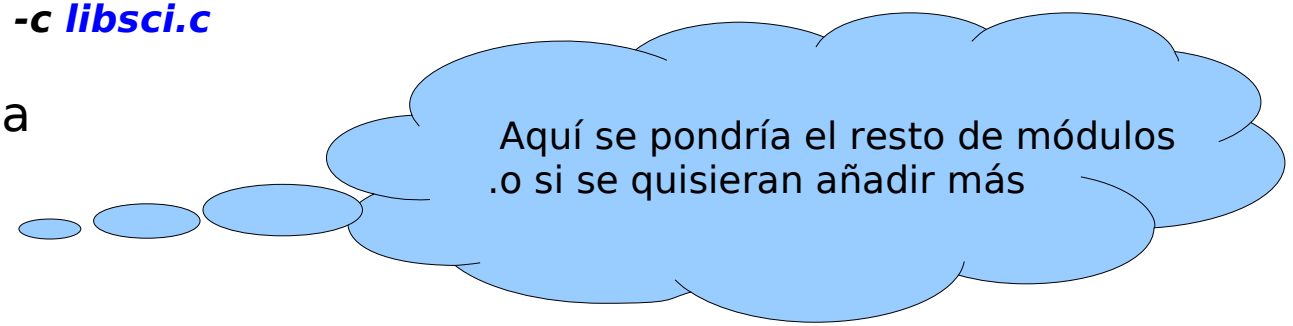
• Vamos a crear una librería de ejemplo, que **SOLO CONTENGA EL MÓDULO libsci** desarrollado anteriormente. La llamamos libsci.lib

• Primero se compila el módulo, para generar el fichero libsci.o

sdcc -mpic14 -p16f876a -c libsci.c

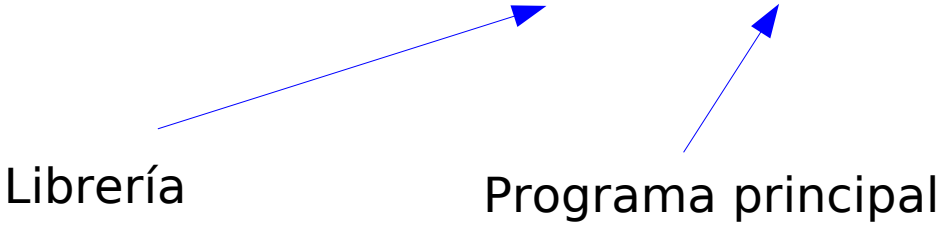
• Segundo: se crea la libreria

gplib -c libsci.lib libsci.o



• Para compilar nuestro programa de ejemplo: sci-test2.c

sdcc -Wl-ainhx8m -mpic14 -p16f876a -o sci-test2 libsci.lib sci-test2.c



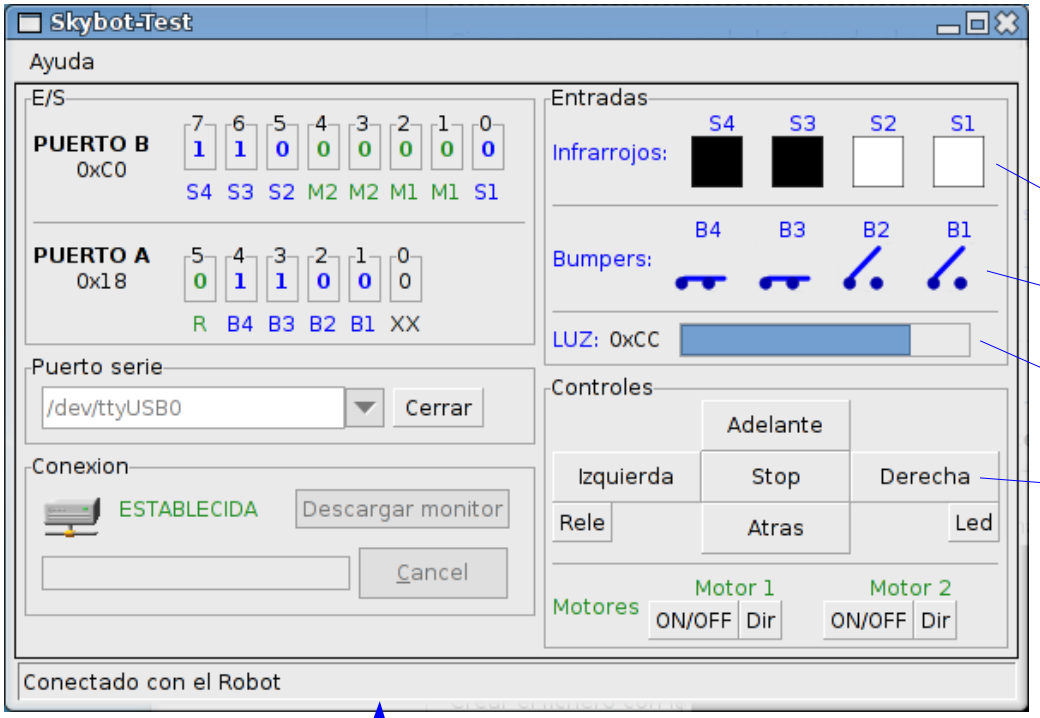
Ejemplo de PIC + PC: Monitorización del Skybot (I)

- El puerto serie es **muy UTIL** para **depurar**
- En el PC se puede usar un terminal de comunicaciones para que el PIC muestre datos
- … Pero también es **muy útil para Monitorizar** y mostrar la información en el PC, usando un programa hecho por nosotros, en vez del terminal (“cutre”).



- Esto ahorra mucho tiempo a la hora de depurar firmware
- Se encuentran fallos en el hardware facilmente
- Se obtiene una interfaz “amigable” con el hardware
- **Ejemplo**: Monitorización y control del robot SKYBOT

Ejemplo de PIC + PC: Monitorización del Skybot (II)



- Programa en el PC para ver estado de los sensores del robotEl

Sensores de infrarrojos

Sensores de contacto

Sensor de luz

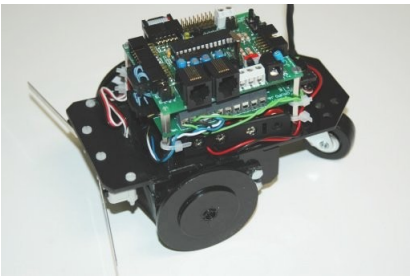
Control de los motores

- Usado en los talleres de robótica
- Detección de problemas con los sensores y motores



PC

Comunicación serie



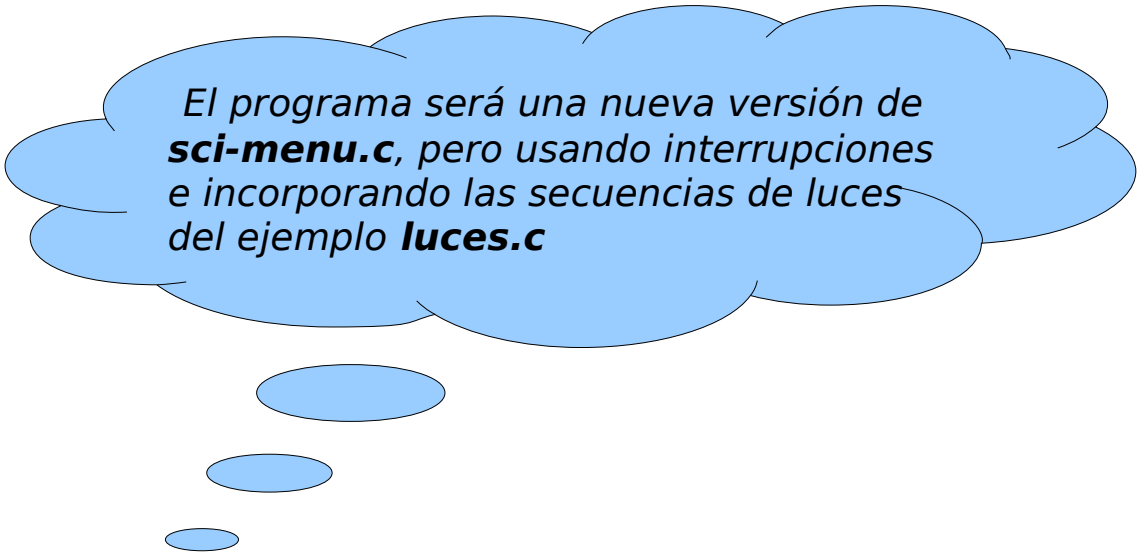
Robot Skybot

Ejercicio final de puerto serie:

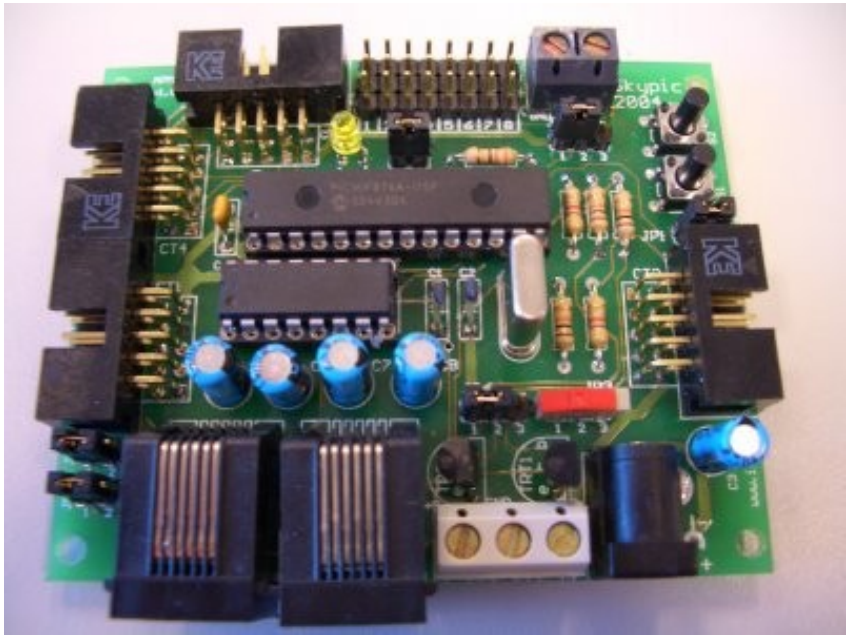
• Hacer un programa para que se muestren **diferentes animaciones** en los leds conectados al **puerto B**

• El programa ofrecerá un **menú al usuario** para poder seleccionar entre **3 secuencias diferentes** al menos

• El usuario en cualquier momento podrá escoger otra secuencia, que se ejecutará inmediatamente al recibir la opción (habrá que utilizar **interrupciones**...)



PIC 16F87X



Juan González

Escuela Politécnica Superior
Universidad Autónoma de Madrid

Andrés Prieto-Moreno

Flir Networked Systems

Ricardo Gómez

Flir Networked Systems